

Face Mask Detection: A Computer Vision Approach for Object Classification

Introduction:

In today's world, ensuring proper face mask usage is a crucial step in maintaining public health and safety, especially in crowded spaces, workplaces, and healthcare environments. To address this need, our project focuses on developing an intelligent system capable of automatically detecting whether a person is wearing a face mask *correctly*, *incorrectly*, or *not at all* by analysing images in real time.

Why This Matters?

Face masks have become an essential tool in preventing the spread of airborne illnesses. However, simply wearing a mask isn't always enough—it must be worn *properly* to be effective. Common mistakes, such as leaving the nose uncovered or wearing the mask loosely, can significantly reduce its protective benefits. Manual monitoring of mask compliance can be time-consuming and inconsistent, which is where our automated solution comes into play.

Our Approach: Leveraging Deep Learning for Accurate Detection

To tackle this challenge, we'll employ Convolutional Neural Networks (CNNs), a powerful deep learning technique widely used in image recognition tasks. CNNs excel at identifying visual patterns, making them ideal for distinguishing between different mask-wearing states.

Here's how our system will work:

1. **Dataset Collection & Preparation:** – We'll gather a diverse set of labelled images featuring people wearing masks correctly, incorrectly, or not at all. This dataset will help the model learn key visual cues, such as mask positioning and facial coverage.
2. **Model Training:** – Using the labelled data, we'll train a CNN to classify each image into one of the three categories. The model will learn to recognize subtle differences, such as a mask below the nose (incorrect) versus one that fully covers the nose and mouth (correct).
3. **Validation & Testing:** – We'll rigorously test the model's accuracy on unseen images to ensure it performs reliably in real-world scenarios.
4. **Deployment :** – Once trained, the system can be integrated into cameras or monitoring systems in public spaces, workplaces, or healthcare facilities to provide instant feedback on mask compliance.

Potential Impact

This technology has far-reaching applications:

- **Public Spaces:**– Airports, malls, and transit hubs can use it to remind individuals to adjust their masks.
- **Workplaces:**– Employers can ensure a safer environment for employees and visitors.
- **Healthcare Settings:**– Hospitals and clinics can minimize infection risks by verifying proper mask usage at entry points.

By automating mask detection, we aim to create a tool that not only enhances safety but also reduces the burden on human monitors, making public health measures more efficient and scalable.

Objective:

Our primary goal is to create a robust system that can:

1. **Detect Faces in Real Time :** – Identify human faces within live video feeds or static images.
2. **Classify Mask Usage :** – Accurately categorize each detected face into one of three classes:
 - Mask Worn Correctly (covering both nose and mouth securely)
 - Mask Worn Incorrectly (e.g., below the nose, loose-fitting, or improperly positioned)
 - No Mask Detected (face fully exposed)
3. **Operate Efficiently :** – Process images quickly enough for real-world applications without significant delays.
4. **Adapt to Different Environments :** – Perform well under varying lighting conditions, angles, and mask types (surgical, cloth, N95, etc.).

Problem Statement:

The manual inspection of mask-wearing behavior is not scalable and is prone to human error. The project proposes a computer vision-based approach that automates this task with higher accuracy and less resource consumption. The main challenge lies in accurately detecting and classifying faces under varying lighting conditions, angles, occlusions, and facial features.

Tools and Technologies Used:

- Python: The Brain of the Operation :-

Python is our go-to programming language because it's versatile and easy to use. It handles everything from loading the training data to running the final detection system. Think of Python as the project manager—it coordinates all the other tools to get the job done.

- OpenCV: The Eyes of the System:-

OpenCV gives our program the ability to "see" and understand images and video. It's responsible for:

- Capturing live video from our webcam or CCTV feed
- Finding faces in the footage (using either classic methods or smarter AI approaches)
- Showing the results by drawing boxes around faces and labeling mask status

- TensorFlow + Keras: The Learning Engine :-

These powerful tools handle all the machine learning heavy lifting:

- Preparing our training data (getting images ready for the AI to learn from)
- Building and training the neural network—this is where our AI actually learns to recognize masks
- Using MobileNetV2 (a pre-trained smart model) to jumpstart the learning process
- Saving the trained knowledge so we can use it again without retraining

- MobileNetV2: The Smart Assistant :-

This pre-trained model is like hiring an expert who already knows a lot about recognizing objects in images. We then give it special training about masks so it becomes even better at this specific task. Its best features:

- Works quickly even on regular computers
- Learns efficiently without needing millions of examples
- Balances accuracy and speed perfectly for real-time detection

- NumPy: The Math Whiz :-

NumPy handles all the number crunching behind the scenes:

- Organizes image data into formats the AI can understand
- Performs lightning-fast calculations that make the whole system responsive
- Manages the data flow between different parts of the program

- Matplotlib: The Visual Storyteller :-

This tool helps us understand how well our AI is learning by creating clear graphs that show:

- Learning progress over time
- Where the model struggles so we can improve it
- Final performance metrics in an easy-to-read format

- Scikit-learn: The Quality Checker :-

Before we deploy our system, we use scikit-learn to thoroughly test it:

- Measures real-world accuracy with precise statistics
- Creates a confusion matrix (a report card showing what the AI gets right and wrong)
- Splits our data properly to ensure fair testing conditions

How They All Work Together :-

- i. OpenCV spots faces in the video feed
- ii. NumPy prepares these faces for analysis
- iii. MobileNetV2 (powered by TensorFlow) decides if there's a mask
- iv. Python coordinates everything and delivers the final result
- v. Matplotlib and scikit-learn help us monitor and improve quality

Dataset Details :-

The dataset used for training this model was sourced from Kaggle, a reputable platform for open-access datasets in machine learning. It consists of a carefully curated collection of labeled facial images, designed to help the model accurately detect and classify mask-wearing behavior.

Key Features of the Dataset

- Three Distinct Classes :-
 - "Mask On": Images of individuals wearing masks correctly, covering both the nose and mouth.
 - "Incorrect Mask": Examples of improper mask usage (e.g., masks below the nose, loose-fitting, or tilted).
 - "No Mask": Clear facial images with no masks present.
- Diversity and Real-World Variability:
 - The dataset includes hundreds of images per category, ensuring robust training.
 - Faces vary in terms of:
 - Lighting conditions (indoor, outdoor, low-light, and bright environments).
 - Angles and perspectives (frontal, side profiles, and tilted heads).
 - Mask types (surgical, cloth, N95, and other common variants).
- Balanced Representation:
 - Each class is well-represented to prevent bias in the model's predictions.
 - Images feature people of different ages, genders, and ethnicities to enhance generalization.

Project Workflow:-

1. Dataset Collection
2. Image Preprocessing
3. Model Building with Transfer Learning
4. Training and Validation
5. Evaluation with Scikit-learn
6. Real-Time Face Detection and Classification
7. Deployment and Testing

Data Preprocessing:-

To prepare our image dataset for training with MobileNetV2, we implement a comprehensive preprocessing pipeline that ensures optimal model performance. Here's a detailed breakdown of each step:

1. Image Resizing

- All input images are uniformly resized to 224x224 pixels, the required input dimensions for MobileNetV2.
- This standardization ensures compatibility with the model's architecture and maintains consistency across the dataset.
- We use bilinear or bicubic interpolation to preserve image quality during resizing, avoiding distortion or excessive pixelation.

2. Pixel Normalization

- Pixel values are scaled to a range of $[-1, 1]$ or $[0, 1]$ (depending on framework conventions) to enhance numerical stability during training.
- Normalization helps the model converge faster by ensuring that gradients remain well-conditioned and prevents any single channel (R, G, B) from dominating due to scale differences.

3. Data Augmentation

To improve generalization and prevent overfitting, we apply the following augmentations randomly during training:

- **Horizontal Flips:** Images are mirrored left-to-right with a 50% probability, simulating different orientations.
- **Brightness Adjustments:** Random brightness shifts ($\pm 20\%$ of the original value) help the model adapt to varying lighting conditions.
- **Rotations:** Images are rotated slightly (e.g., ± 15 degrees) to account for minor orientation changes in real-world scenarios.
- *(Optional: Additional augmentations like slight zooms, translations, or contrast adjustments can be included for further robustness.)*

Augmentations are applied on-the-fly during training, meaning each batch contains slightly modified versions of the original images, effectively expanding the dataset's diversity.

4. Train-Test Split

- The dataset is divided into 80% training and 20% testing subsets using a stratified split (if labels are imbalanced) to maintain class distribution.
- Before splitting, we shuffle the data to eliminate any inherent ordering bias.
- The test set remains completely untouched by augmentation, ensuring a fair evaluation of the model's real-world performance.

Implementation Notes

- Efficiency: Resizing and normalization are applied once (preprocessed), while augmentations are performed dynamically during training (e.g., using TensorFlow's ImageDataGenerator or PyTorch's transforms).
- Reproducibility: Random operations (augmentation, shuffling) use fixed seeds where necessary for debugging or comparison.

Model Architecture

To build an effective image classification system, we leverage MobileNetV2 as our foundational feature extractor, combined with a set of custom layers designed to refine predictions for our specific 3-class problem. Below is a detailed breakdown of the model structure and training setup:

1. Model Architecture

Base Model: MobileNetV2 (Pre-trained on ImageNet)

- Purpose: Acts as a powerful feature extractor, capturing essential patterns from images without requiring training from scratch.
- Configuration:
 - Input shape: 224x224x3 (RGB images, resized during preprocessing).
 - Weights: Pre-trained on ImageNet to utilize learned hierarchical features.
 - Fine-tuning: The base model can be frozen (initially) to prevent large weight updates, or selectively unfrozen for later fine-tuning.

2. Custom Classification Head

To adapt MobileNetV2 for our specific task, we stack the following layers:

- Global Average Pooling 2D
 - Reduces spatial dimensions (H x W) to a single vector by averaging each feature map.
 - Helps minimize overfitting compared to Flatten layers while preserving spatial information.

- Dense Layer (ReLU Activation, 128–256 units)
 - Introduces learnable parameters for high-level feature interpretation.
 - ReLU activation introduces non-linearity, enabling complex decision boundaries.
- Dropout (Rate: 0.2–0.5)
 - Randomly deactivates a fraction of neurons during training (e.g., 20–50%) to prevent over-reliance on specific features.
 - Acts as a regularizer, improving generalization to unseen data.
- Output Layer (Softmax, 3 Units)
 - Produces probability distributions across the 3 classes.
 - Softmax activation ensures outputs sum to 1, interpretable as class confidence scores.

3. Training Parameters

- Loss Function: Categorical Crossentropy
 - Measures the discrepancy between predicted probabilities and true one-hot encoded labels.
 - Ideal for multi-class classification where classes are mutually exclusive.
- Optimizer: Adam
 - Combines momentum (like RMSProp) and adaptive learning rates for efficient convergence.
 - Default parameters ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-7$) typically suffice, but learning rate can be tuned if needed.
- Batch Size: 32
 - Balances memory efficiency and gradient stability.
 - Smaller batches introduce noise (helping generalization), while larger batches offer smoother updates.
- Epochs: 20
 - The model processes the entire training set 20 times.
 - Early stopping may be implemented (e.g., halting if validation loss plateaus for 3–5 epochs) to avoid overfitting.

Training the Model :-

To ensure our image classifier achieves both accuracy and robustness, we carefully design the training process using TensorFlow and Keras, leveraging transfer learning to capitalize on pre-trained knowledge while adapting to our specific task. Below is a comprehensive breakdown of the training workflow:

1. Transfer Learning Setup

Why MobileNetV2?

- MobileNetV2 is a lightweight yet powerful architecture, pre-trained on ImageNet, which has already learned to recognize a wide array of visual features (edges, textures, shapes).
- By using it as a fixed feature extractor (initially freezing its weights), we avoid training from scratch, saving time and computational resources.

Freezing vs. Fine-Tuning

- Phase 1 (Feature Extraction):
 - The base MobileNetV2 layers are frozen—their weights remain static.
 - Only the custom head (GlobalAveragePooling2D, Dense, Dropout) is trained.
 - This phase focuses on adapting the high-level layers to our specific classes.
- Phase 2 (Optional Fine-Tuning):
 - After initial training, the top layers of MobileNetV2 can be unfrozen.
 - The entire model is trained end-to-end with a very low learning rate (e.g., $1e-5$) to subtly refine features.
 - Fine-tuning is delicate; overdoing it may lead to overfitting.

2. Training Execution

- Monitoring Key Metrics
 - Training Loss (Categorical Crossentropy): Tracks how well the model fits the training data. A steady decrease indicates healthy learning.
 - Validation Loss: Measures generalization. If validation loss rises while training loss falls, overfitting is likely.
 - Validation Accuracy: The primary benchmark for model performance. We aim for consistent improvement.
- Critical Safeguards

- Early Stopping:
 - Halts training if validation loss fails to improve for a set number of epochs (e.g., 5).
 - Prevents wasted epochs and overfitting.
- Model Checkpointing:
 - Saves the model weights only when validation accuracy improves.
 - Ensures we retain the best version for deployment.
- Learning Rate Scheduling:
 - Reduces the learning rate if validation metrics plateau (e.g., by a factor of 0.1).
 - Helps the model settle into optimal minima.
- Hardware Considerations
 - GPU Acceleration: Training is significantly faster on GPUs (e.g., NVIDIA CUDA cores).
 - Batch Size Trade-offs: A batch size of 32 balances memory usage and gradient stability. Larger batches may speed up training but reduce generalization.

3. Post-Training Validation

- Confusion Matrix: Reveals per-class performance (precision, recall) to identify weak points (e.g., a class consistently misclassified).
- Error Analysis: Manual inspection of misclassified images helps diagnose issues (e.g., poor lighting, ambiguous labels).
- Test Set Evaluation: Final metrics on the held-out test set (20% of data) provide an unbiased performance estimate.

4. Deployment Preparation

- Model Export: The best-performing model is saved in a format suitable for deployment (e.g., .h5 or TensorFlow SavedModel).
- Optimizations (Optional):
 - Quantization: Reduces model size for edge devices (e.g., TFLite for mobile).
 - Pruning: Trims insignificant neurons to improve efficiency.
- Real-Time Testing: Deployed model is tested in a staging environment with synthetic or real-world inputs to catch edge cases.

Why This Approach Works

- Efficiency: Transfer learning drastically cuts training time and data requirements.
- Reliability: Monitoring validation metrics ensures the model generalizes, not just memorizes.
- Scalability: The same pipeline can adapt to new classes or datasets with minimal tweaks.

Evaluation Metrics :-

To thoroughly assess the performance of our trained MobileNetV2 classifier, we leverage Scikit-learn's powerful evaluation metrics, which provide deep insights into how well the model generalizes across all three categories. Below is an expanded breakdown of each metric, its significance, and how it helps us refine the model for real-world reliability.

1. Confusion Matrix: The Foundation of Diagnostic Analysis

What It Shows

- A 3x3 matrix (since we have 3 classes) that compares predicted labels against true labels.
- Each cell $[i][j]$ indicates how many instances of Class i were predicted as Class j .

Why It Matters

- Diagonal cells (True Positives): Correct predictions (ideal: high values).
- Off-diagonal cells (Errors): Misclassifications reveal model weaknesses (e.g., Class A often confused with Class B).
- Actionable Insights:
 - If two classes are frequently mixed up, we might need:
 - More training data for those classes.
 - Adjustments to augmentation (e.g., rotations/flips to reduce bias).
 - Fine-tuning the model's decision boundaries.

2. Accuracy: The Big-Picture Metric

Calculation

- $\text{Accuracy} = (\text{True Positives} + \text{True Negatives}) / \text{Total Samples}$
- Measures overall correctness across all classes.

Caveats

- Can be misleading for imbalanced datasets (e.g., if 90% of samples are Class A, a model guessing "A" blindly achieves 90% accuracy).
- Best Used With: Other metrics (precision, recall) for a balanced view.

3. Precision: Measuring Exactness

Calculation

- $\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$
- Answers: *"When the model predicts Class X, how often is it correct?"*

Interpretation

- High Precision: Few false alarms (critical for applications like medical diagnosis, where false positives are costly).
- Low Precision: The model is overly optimistic, frequently mislabeling other classes as the target class.

4. Recall (Sensitivity): Measuring Completeness

Calculation

- $\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$
- Answers: *"What fraction of actual Class X instances did the model catch?"*

Interpretation

- High Recall: The model misses few true positives (vital for safety-critical tasks, like defect detection).
- Low Recall: The model overlooks too many real cases (e.g., failing to identify a disease in patients).

5. F1-Score: The Harmonic Balance

Calculation

- $F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$
- A single metric balancing precision and recall.

Why It's Critical

- High F1: The model achieves both low false positives *and* low false negatives.
- Low F1: Flags a trade-off issue (e.g., high precision but poor recall, or vice versa).
- Class-Specific: Calculate F1 per class to identify which categories struggle.

Real-Time Detection Flow :-

To create an intuitive and effective real-time mask detection system, we combine computer vision and deep learning into a seamless workflow.

1. Capturing Live Video with OpenCV

How It Works

- The system accesses the device's camera (webcam or smartphone) using OpenCV's VideoCapture.
- Frame-by-frame processing: Each snapshot of the video is treated as an individual image for analysis.
- Efficiency tweaks:
 - Resolution can be adjusted (e.g., 720p) to balance speed and detail.
 - Frames are read at 20–30 FPS for smooth real-time feedback.

User Experience Considerations

- Low-latency: Minimizes delay between real-world actions and on-screen results.
- Privacy-first: No video data is stored; processing happens live in memory.

2. Face Detection with Haar Cascades

Why Haar Cascades?

- Lightweight and fast for real-time use, ideal for edge devices (e.g., Raspberry Pi).
- Trained to detect frontal and profile faces, even in varied lighting.

The Detection Process

- Grayscale conversion: Simplifies analysis while preserving structural features.
- Multi-scale detection: Searches for faces at different sizes (small faces far away, large faces up close).
- False positive filtering: Overlapping detections are merged, and low-confidence regions are discarded.

Limitations & Mitigations

- Challenge: Struggles with extreme angles or occlusions (e.g., faces turned away).
- Solution: Fallback to more robust detectors (e.g., DNN-based models like YOLO or MTCNN) if Haar cascades miss faces.

3. Preprocessing Detected Faces

Standardization for the Model

- Resizing: Faces are cropped and resized to 224x224 pixels, matching MobileNetV2's input requirements.
- Normalization: Pixel values are scaled to [0, 1] or [-1, 1] (consistent with training).
- Quality checks:
 - Blurry or extremely small faces (<50x50 pixels) are rejected.
 - Brightness normalization compensates for poor lighting.

4. Mask Status Prediction

Model Inference

- The preprocessed face is fed into our fine-tuned MobileNetV2 model.
- Outputs a 3-class probability distribution (e.g., [No Mask: 0.1, Mask: 0.8, Improper Mask: 0.1]).
- Decision rule: The class with the highest probability is selected (with a confidence threshold, e.g., >80%).

Edge Cases Handled

- Uncertain predictions: Low-confidence results trigger a "Check Mask" warning instead of a definitive label.
- Multiple faces: Each face is processed independently, with parallel predictions where possible.

5. Displaying Results with OpenCV

User-Friendly Annotations

- Bounding boxes: Drawn around detected faces with colors indicating status:
 - Green: Mask worn correctly.
 - Yellow: Mask improperly worn (e.g., below nose).
 - Red: No mask detected.
- Text labels: Clear annotations (e.g., "MASK: 95% CONFIDENCE").
- Real-time feedback: Audio alerts (optional) can reinforce visual cues for accessibility.

Performance Optimizations

- Asynchronous processing: While one frame is analyzed, the next is captured to maintain FPS.
- Hardware acceleration: OpenCV leverages GPUs or NPUs (if available) for faster rendering.

Algorithm Justification:-

Neural Networks (CNN with MobileNetV2) is chosen for the following reasons:

- Automatically learns complex visual patterns
- Efficient feature extraction
- Performs well with limited labeled data through transfer learning
- Achieves real-time performance with MobileNetV2's lightweight structure

Challenges Faced:-

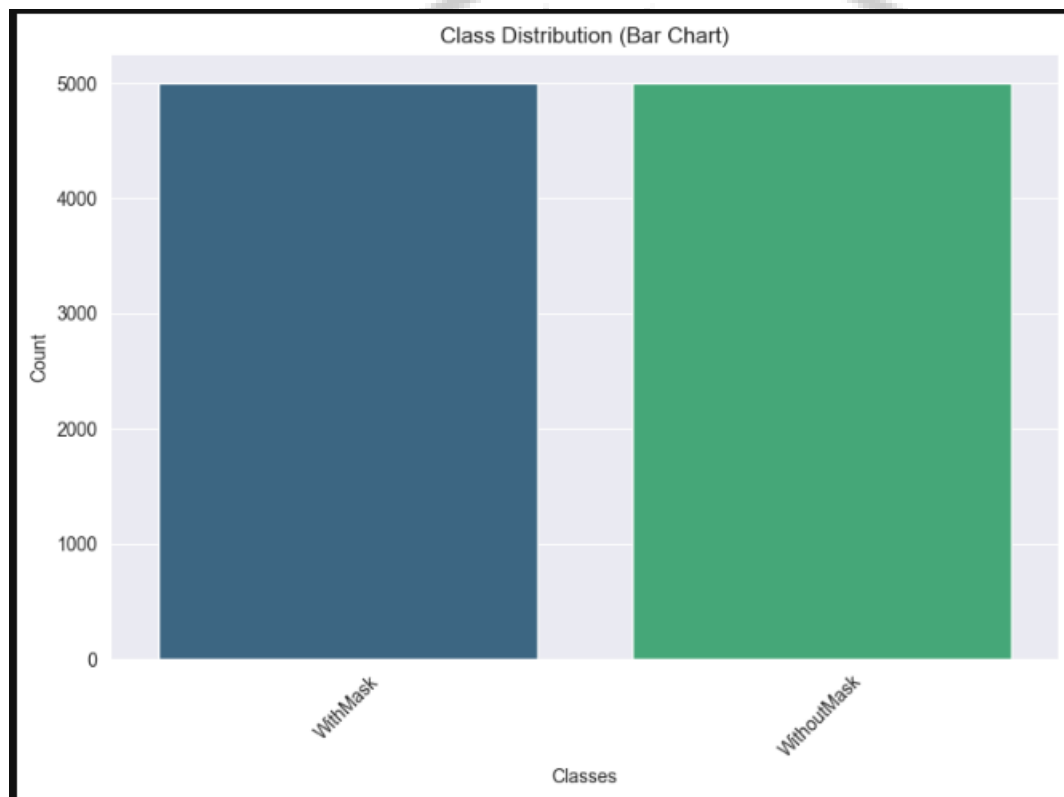
- Mask Variability
 - Problem: Masks come in all shapes (N95, cloth, surgical) and colors, some with prints or logos.
 - Solution: Train on diverse datasets—include unconventional masks, and augment with synthetic patterns.
- Partial Mask Wear
 - Problem: "Mask on chin" or "below nose" defeats the purpose but confuses basic models.
 - Solution: Add an "improper" class during training, using cropped/nose-exposed face images.
- Small or Biased Datasets
 - Problem: Limited labeled data leads to poor generalization.
 - Solution: Use transfer learning (MobileNetV2 pre-trained weights) and aggressive augmentation (rotations, shadows, occlusions).
- Performance on Budget Hardware
 - Problem: Heavy models lag on low-end devices.

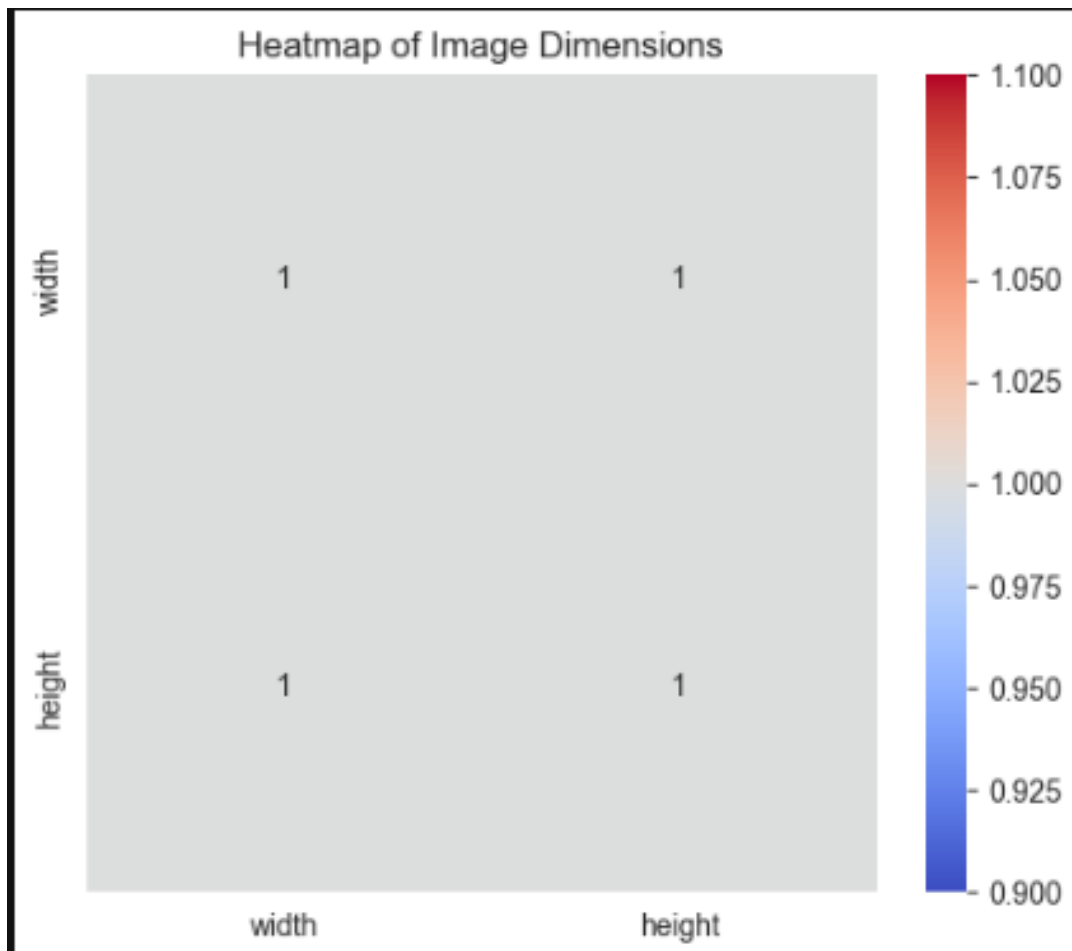
- Solution: Optimize with TensorFlow Lite, reduce input resolution (160x160), or cap frame rate to 15 FPS.

Solutions Implemented :-

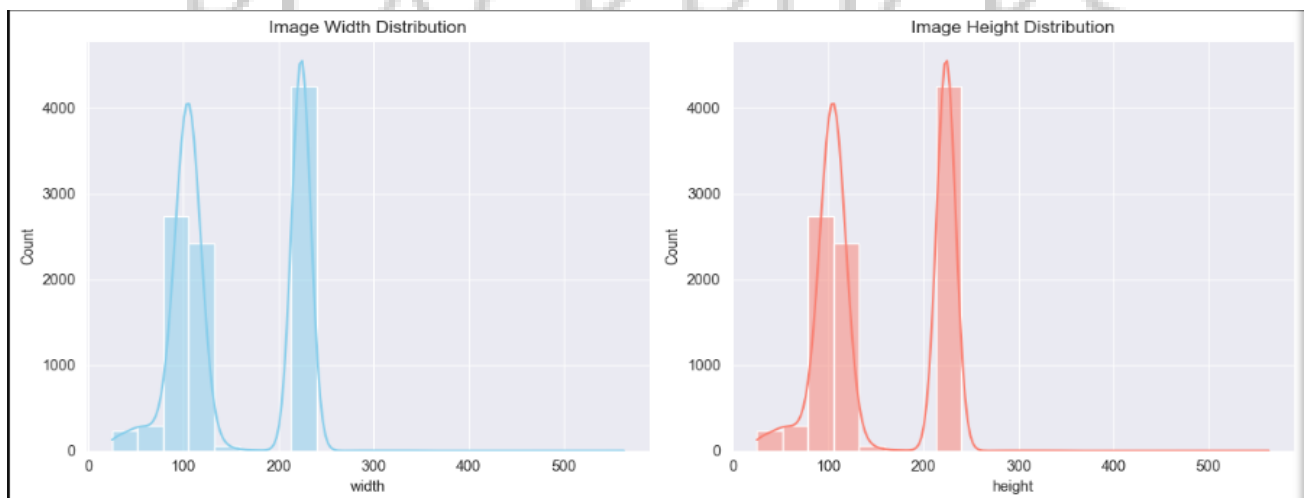
- Data augmentation for diversity
- Using pre-trained MobileNetV2 for transfer learning
- Fine-tuning on mask-related data
- Lightweight model architecture for speed

Output:-

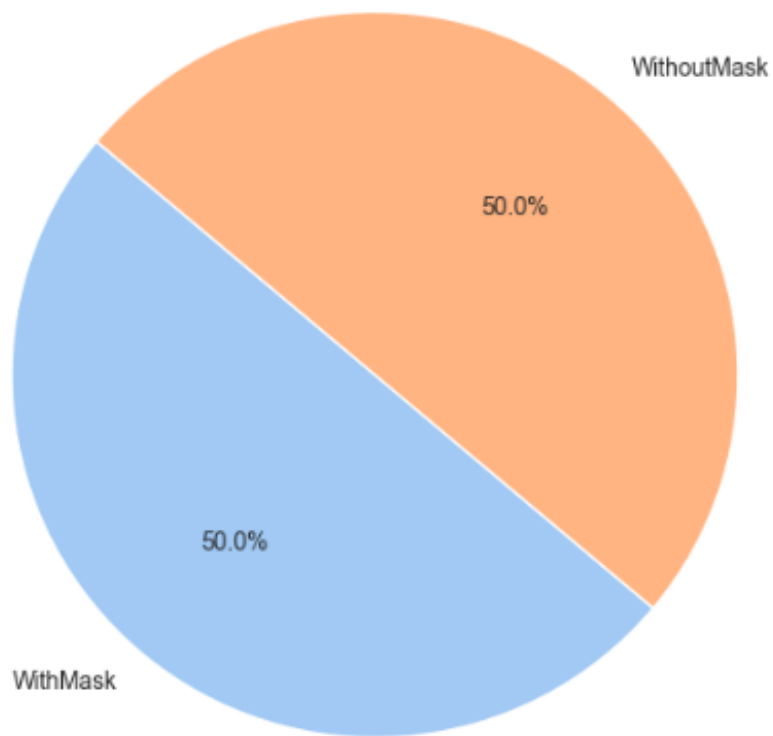




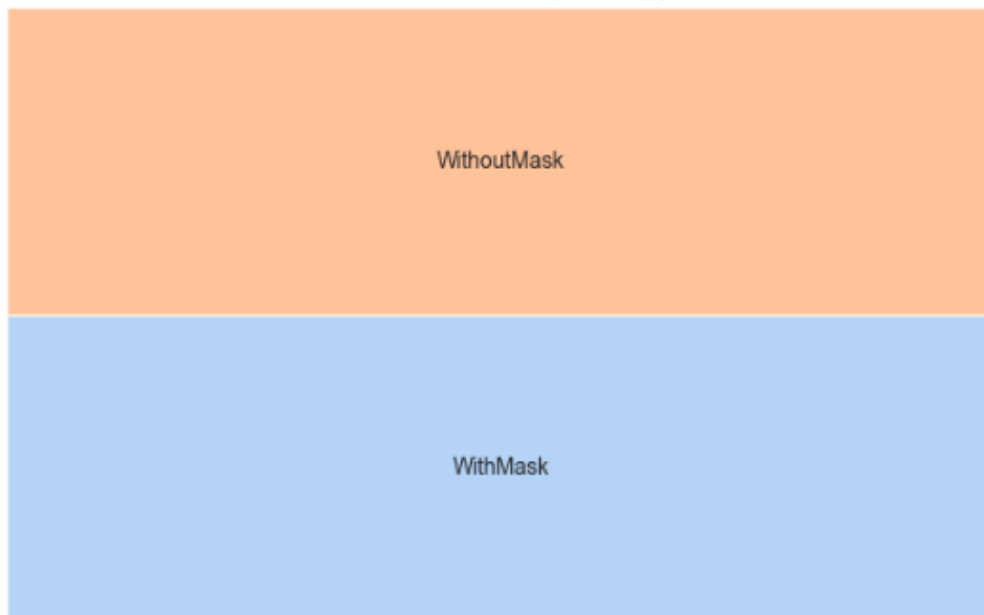
BLACKBLICKS



Class Distribution (Pie Chart)



Class Distribution (Treemap)



Model: "sequential"

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 1280)	4049571
batch_normalization (Batch Normalization)	(None, 1280)	5120
dense (Dense)	(None, 256)	327936
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 2)	514

=====
Total params: 4383141 (16.72 MB)

Trainable params: 331010 (1.26 MB)

Non-trainable params: 4052131 (15.46 MB)

Epoch 1/5

WARNING:tensorflow:From C:\Users\Adrika\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.nn.conv2d is deprecated. Please use tf.nn.conv2d_v2 instead.

625/625 [=====] - 391s 606ms/step - loss: 5.6294 - accuracy: 0.5162 - val_loss: 3.9556 - val_accuracy: 0.6134 - lr: 5.0000e-04

Epoch 2/5

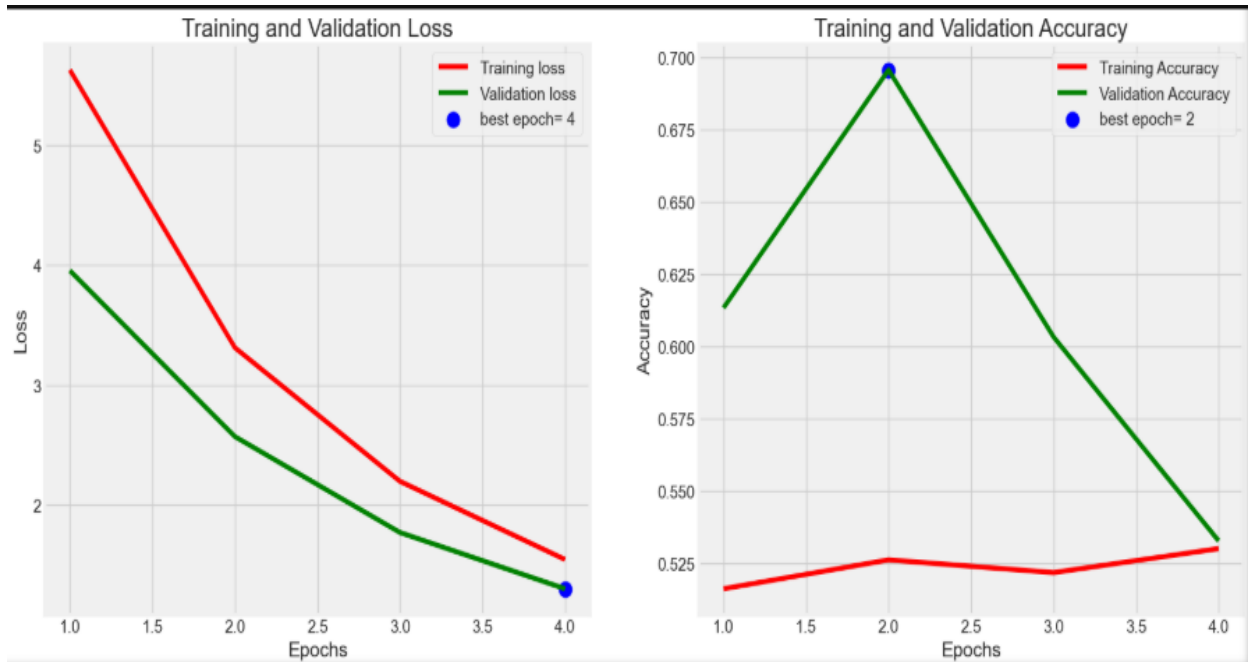
625/625 [=====] - 374s 598ms/step - loss: 3.3122 - accuracy: 0.5262 - val_loss: 2.5706 - val_accuracy: 0.6958 - lr: 5.0000e-04

Epoch 3/5

625/625 [=====] - 373s 596ms/step - loss: 2.1975 - accuracy: 0.5218 - val_loss: 1.7706 - val_accuracy: 0.6034 - lr: 5.0000e-04

Epoch 4/5

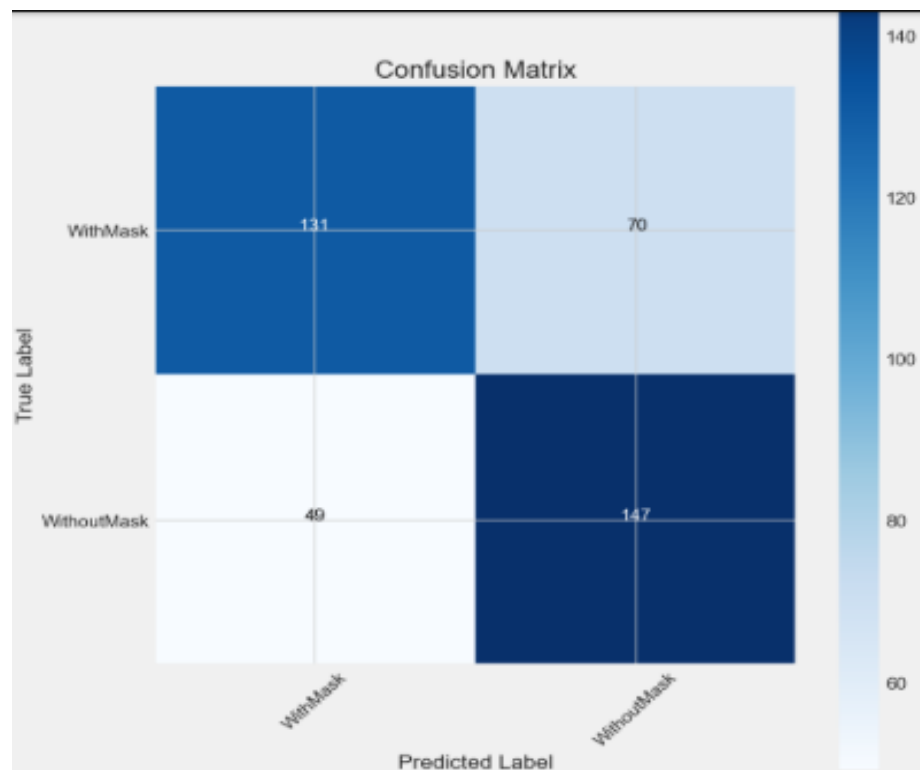
625/625 [=====] - 370s 592ms/step - loss: 1.5453 - accuracy: 0.5301 - val_loss: 1.3012 - val_accuracy: 0.5328 - lr: 5.0000e-04



```

397/397 [=====] - 223s 561ms/step - loss: 2.5703 - accuracy: 0.6836
38/397 [=>.....] - ETA: 3:21 - loss: 2.5709 - accuracy: 0.6824WARNING:tensorflow:Your input ran out of data; interrupt
ing training. Make sure that your dataset or generator can generate at least `steps_per_epoch * epochs` batches (in this case, 397 batches). Y
ou may need to use the repeat() function when building your dataset.
397/397 [=====] - 22s 53ms/step - loss: 2.5709 - accuracy: 0.6824
25/397 [>.....] - ETA: 3:35 - loss: 2.5694 - accuracy: 0.6851WARNING:tensorflow:Your input ran out of data; interrupt
ing training. Make sure that your dataset or generator can generate at least `steps_per_epoch * epochs` batches (in this case, 397 batches). Y
ou may need to use the repeat() function when building your dataset.
397/397 [=====] - 15s 35ms/step - loss: 2.5694 - accuracy: 0.6851
Train Loss: 2.570335626602173
Train Accuracy: 0.6835642457008362
-----
Validation Loss: 2.5708744525909424
Validation Accuracy: 0.6823529601097107
-----
Test Loss: 2.569352865219116
Test Accuracy: 0.6851385235786438

```



	precision	recall	f1-score	support
WithMask	0.73	0.65	0.69	201
WithoutMask	0.68	0.75	0.71	196
accuracy			0.70	397
macro avg	0.70	0.70	0.70	397
weighted avg	0.70	0.70	0.70	397

Conclusion:-

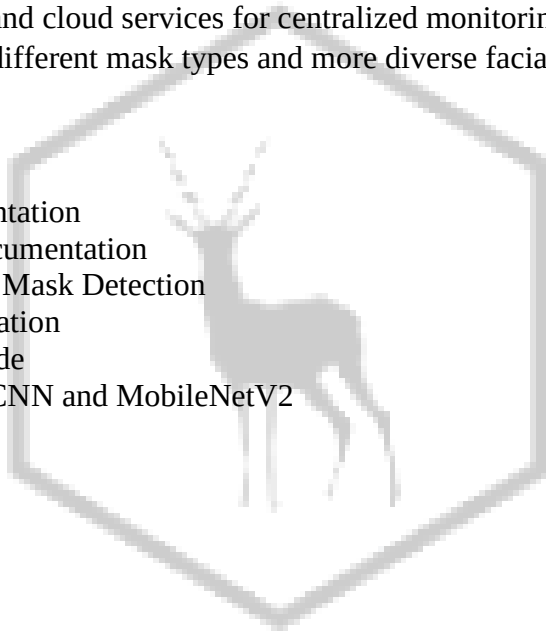
This project demonstrates how artificial intelligence and computer vision can contribute to public health monitoring. By combining OpenCV for image capture and TensorFlow with MobileNetV2 for classification, a robust and real-time face mask detection system has been built. It is scalable, efficient, and ready for deployment in real-world environments.

Future Scope :-

- Extend detection to thermal screening and social distancing monitoring.
- Integration with IoT and cloud services for centralized monitoring.
- Expand dataset with different mask types and more diverse facial images.

References :-

- TensorFlow Documentation
- OpenCV Official Documentation
- Kaggle Dataset: Face Mask Detection
- Keras API Documentation
- Scikit-learn User Guide
- Research articles on CNN and MobileNetV2



BLACKBUCKS
GROUP OF COMPANIES